

Threads – Java

**Programacion Concurrente
2025**

Threads – Ejemplo

Ejemplo

```
int y1 = 0;  
int y2 = 0;  
short in_critical = 0;
```

```
active proctype process_1() {  
  do  
    :: true ->  
      y1 = y2+1;  
      ((y2==0) || (y1<=y2));  
      in_critical++;  
      in_critical--;  
      y1 = 0;  
  od  
}
```

```
active proctype process_2() {  
  do  
    :: true ->  
      y2 = y1+1;  
      ((y1==0) || (y2<y1));  
      in_critical++;  
      in_critical--;  
      y2 = 0;  
  od  
}
```

Threads – Ejemplo

```
public class Main {  
    public static void main(String[] args) {  
        new Proceso();  
    }  
}
```

Threads – Ejemplo

```
public class Proceso {  
  
    public Proceso() {  
  
        T1 tarea1 = new T1();  
        Thread t1 = new Thread(tarea1);  
        t1.start();  
  
        T2 tarea2 = new T2();  
        Thread t2 = new Thread(tarea2);  
        t2.start();  
  
    }  
  
}  
  
public abstract class Tarea implements Runnable {  
    protected static int y1=0,y2=0;  
    protected static int critical=0;  
  
}
```

Threads – Ejemplo

```
public class T1 extends Tarea {

    public T1() {
    }

    public void run() {
        while (true) {
            Tarea.y1 = Tarea.y2 + 1;

            while ((!(Tarea.y2 == 0) && !(Tarea.y1 <= Tarea.y2))) {
            }

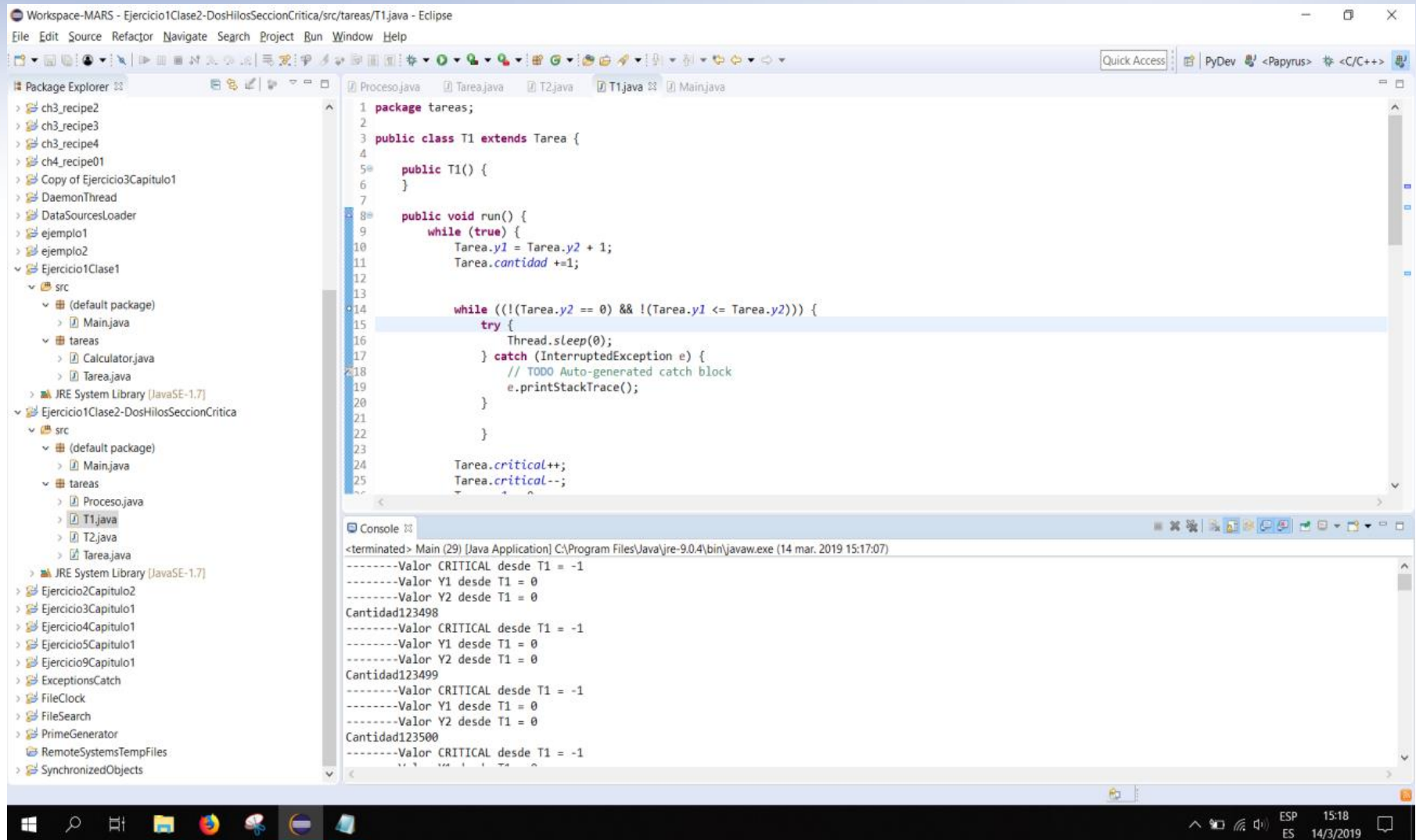
            Tarea.critical++;
            Tarea.critical--;
            Tarea.y1 = 0;

            if (Tarea.critical != 0){
                System.out.println("Valor CRITICAL desde T1 = " + (Tarea.critical));
                System.out.println("Valor Y1 desde T1 = " + (Tarea.y1));
                System.out.println("Valor Y2 desde T1 = " + (Tarea.y2));
            }
        }
    }
}
```

Threads – Ejemplo

```
public class T2 extends Tarea {  
  
    public T2() {  
    }  
  
    public void run() {  
        while (true) {  
            Tarea.y2 = Tarea.y1 + 1;  
  
            while ((!(Tarea.y1 == 0) && !(Tarea.y2 <= Tarea.y1))) {  
            }  
  
            Tarea.critical++;  
            Tarea.critical--;  
  
            Tarea.y2 = 0;  
  
            if (Tarea.critical != 0){  
                System.out.println("Valor CRITICAL desde T2 = " + (Tarea.critical));  
                System.out.println("Valor Y1 desde T2 = " + (Tarea.y1));  
                System.out.println("Valor Y2 desde T2 = " + (Tarea.y2));  
            }  
        }  
    }  
}
```

Threads – Ejemplo - Ejecución



Workspace-MARS - Ejercicio1Clase2-DosHilosSeccionCritica/src/tareas/T1.java - Eclipse

File Edit Source Refactor Navigate Search Project Run Window Help

Quick Access PyDev <Papyrus> <C/C++>

Package Explorer

- ch3_recipe2
- ch3_recipe3
- ch3_recipe4
- ch4_recipe01
- Copy of Ejercicio3Capitulo1
- DaemonThread
- DataSourcesLoader
- ejemplo1
- ejemplo2
- Ejercicio1Clase1
 - src
 - (default package)
 - Main.java
 - tareas
 - Calculator.java
 - Tarea.java
- JRE System Library [JavaSE-1.7]
- Ejercicio1Clase2-DosHilosSeccionCritica
 - src
 - (default package)
 - Main.java
 - tareas
 - Proceso.java
 - T1.java
 - T2.java
 - Tarea.java
- JRE System Library [JavaSE-1.7]
- Ejercicio2Capitulo2
- Ejercicio3Capitulo1
- Ejercicio4Capitulo1
- Ejercicio5Capitulo1
- Ejercicio9Capitulo1
- ExceptionsCatch
- FileClock
- FileSearch
- PrimeGenerator
- RemoteSystemsTempFiles
- SynchronizedObjects

```
1 package tareas;
2
3 public class T1 extends Tarea {
4
5     public T1() {
6     }
7
8     public void run() {
9         while (true) {
10             Tarea.y1 = Tarea.y2 + 1;
11             Tarea.cantidad +=1;
12
13
14             while ((!(Tarea.y2 == 0) && !(Tarea.y1 <= Tarea.y2))) {
15                 try {
16                     Thread.sleep(0);
17                 } catch (InterruptedException e) {
18                     // TODO Auto-generated catch block
19                     e.printStackTrace();
20                 }
21             }
22
23             Tarea.critical++;
24             Tarea.critical--;
25         }
26     }
27 }
```

Console

<terminated> Main (29) [Java Application] C:\Program Files\Java\jre-9.0.4\bin\javaw.exe (14 mar. 2019 15:17:07)

```
-----Valor CRITICAL desde T1 = -1
-----Valor Y1 desde T1 = 0
-----Valor Y2 desde T1 = 0
Cantidad123498
-----Valor CRITICAL desde T1 = -1
-----Valor Y1 desde T1 = 0
-----Valor Y2 desde T1 = 0
Cantidad123499
-----Valor CRITICAL desde T1 = -1
-----Valor Y1 desde T1 = 0
-----Valor Y2 desde T1 = 0
Cantidad123500
-----Valor CRITICAL desde T1 = -1
```

Threads – Ejemplo - Ejecución

The screenshot displays the Eclipse IDE interface. The Package Explorer on the left shows a project structure with a package named 'tareas' containing files 'Proceso.java', 'Tarea.java', 'T1.java', and 'T2.java'. The main editor shows the code for 'T2.java', which defines a class 'T2' extending 'Tarea'. The 'run()' method contains a 'while' loop that increments 'Tarea.y1' and 'Tarea.cantidad' until 'Tarea.y1' reaches 3. The console at the bottom shows the output of the program, indicating that the thread 'T2' has completed its execution and printed the final values of 'Tarea.y1' and 'Tarea.cantidad'.

```
1 package tareas;
2
3 public class T2 extends Tarea {
4
5     public T2() {
6     }
7
8     public void run() {
9         while (true) {
10             Tarea.y1 = Tarea.y1 + 1;
11             Tarea.cantidad +=1;
12
13             while ((!(Tarea.y1 == 0) && !(Tarea.y2 <= Tarea.y1))) {
14                 //System.out.println("-----Trabado desde T2 = ");
15                 try {
16                     Thread.sleep(0);
17                 } catch (InterruptedException e) {
18                     // TODO Auto-generated catch block
19                     e.printStackTrace();
20                 }
21             }
22
23             Tarea.critical++;
24
25             try {
```

Console Output:

```
Main (29) [Java Application] C:\Program Files\Java\jre-9.0.4\bin\javaw.exe (14 mar. 2019 15:24:30)
-----Valor CRITICAL desde T1 = -3
-----Valor Y1 desde T1 = 0
-----Valor Y2 desde T1 = 0
Cantidad491503
-----Valor CRITICAL desde T1 = -3
-----Valor Y1 desde T1 = 0
-----Valor Y2 desde T1 = 0
Cantidad491504
-----Valor CRITICAL desde T1 = -3
-----Valor Y1 desde T1 = 0
-----Valor Y2 desde T1 = 0
Cantidad491505
```


Threads – Creación en Java

Cuando hablamos de concurrencia en el mundo de los procesadores, nos referimos a tareas independientes que se ejecutan simultáneamente en una computadora. Esta simultaneidad puede ser real, si los recursos lo permiten.

- Los hilos son objetos. Hay dos formas de crear hilos en JAVA:
 - Extendiendo la clase Thread y sobrescribiendo el método run().
 - Creando una clase que implemente la interfaz **runnable**. Luego crear un objeto de la clase Thread y pasarle el objeto runnable como argumento.

De estas dos maneras podemos **"darle vida"** a una clase.

Threads - Thread Safe

Característica que brinda la posibilidad de ejecutar el código en un entorno con múltiples hilos garantizando que el recurso no se corrompe.

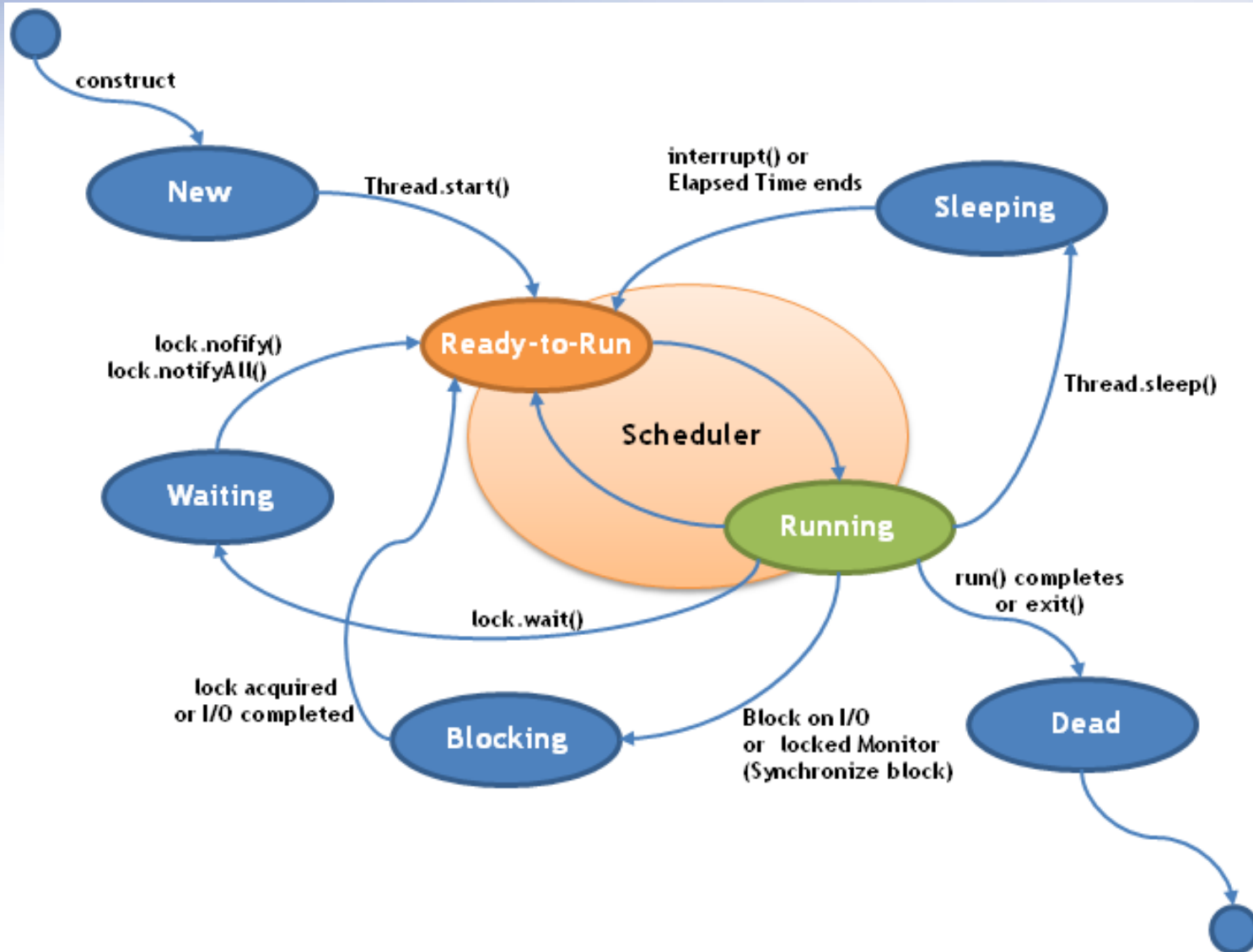
Los atributos *static* son Thread Safe?

Threads - Atributos

La clase Thread, almacena atributos de informacion que pueden ayudarnos a identificar a un thread, conocer su estado, o controlar su prioridad. Estos atributos son:

- **Id:** Este atributo almacena un identificador único para cada thread.
- **Nombre:** Este atributo almacena el nombre del hilo.
- **Prioridad:** Este atributo almacena la prioridad de los objetos hilo.
 - La prioridad en los hilos varia desde 1 hasta 10. 1 es la mas baja. 10 la más alta. Cambiar la prioridad de los hilos no es recomendable.
- **Estado:** Este atributo almacena el estado del hilo. En java los estados posibles de un hilo son 7.

Threads - Estados



Threads - Ejemplo

- Se implementará el proyecto paso a paso:

1. Crear una clase llamada **Calculator**, que implemente la interfaz “runnable”.

```
public class Calculator implements Runnable {
```

2. Implemente el método run. En éste método, el cual sera ejecutado por los hilos creados se calcularán y contarán números primos hasta un determinado límite.

Threads - Ejemplo

```
@Override
public void run() {
    long current = 1L;
    long max = 200L; //pocos numeros que pasa con prioridad? <200
    long numPrimes = 0L;

    System.out.printf("Thread '%s': START\n", Thread.currentThread().getName());
    while (current <= max) {
        if (isPrime(current)) {
            numPrimes++;
        }
        current++;
    }
    System.out.printf("Thread '%s': END. Number of Primes: %d\n",
        Thread.currentThread().getName(), numPrimes);
}
```

Threads - Ejemplo

```
private boolean isPrime(long number) {  
    if (number <= 2) {  
        return true;  
    }  
    for (long i = 2; i < number; i++) {  
        if ((number % i) == 0) {  
            return false;  
        }  
    }  
    return true;  
}
```

Threads - Ejemplo

```
// Thread priority infomation
System.out.printf("Minimum Priority: %s\n", Thread.MIN_PRIORITY);
System.out.printf("Normal Priority: %s\n", Thread.NORM_PRIORITY);
System.out.printf("Maximun Priority: %s\n", Thread.MAX_PRIORITY);
```

Luego se crean: 10 hilos para ejecutar los 10 objetos tarea; se crean además dos arreglos para almacenar los objetos threads y sus estados. Se setea el atributo prioridad de los hilos pares en máximo y los impares en mínimo.

Threads - Ejemplo

```
Thread threads[];  
Thread.State status[];  
  
// Launch 10 threads to do the operation, 5 with the max  
// priority, 5 with the min  
threads = new Thread[10];  
status = new Thread.State[10];  
for (int i = 0; i < 10; i++) {  
    threads[i] = new Thread(new Calculator());  
    if ((i % 2) == 0) {  
        threads[i].setPriority(Thread.MAX_PRIORITY);  
    } else {  
        threads[i].setPriority(Thread.MIN_PRIORITY);  
    }  
    threads[i].setName("My Thread " + i);  
}
```

Threads - Ejemplo

```
// Wait for the finalization of the threads. Meanwhile,  
// write the status of those threads in a file  
try (FileWriter file = new FileWriter(".\\data\\log.txt");  
     PrintWriter pw = new PrintWriter(file);) {  
  
    // Write the status of the threads  
    for (int i = 0; i < 10; i++) {  
        pw.println("Main : Status of Thread " + i + " : " + threads[i].getState());  
        status[i] = threads[i].getState();  
    }  
  
    // Start the ten threads  
    for (int i = 0; i < 10; i++) {  
        threads[i].start();  
    }  
}
```

Threads - Ejemplo

```
// Wait for the finalization of the threads. We save the status of
// the threads and only write the status if it changes.
boolean finish = false;
while (!finish) {
    for (int i = 0; i < 10; i++) {
        if (threads[i].getState() != status[i]) {
            writeThreadInfo(pw, threads[i], status[i]);
            status[i] = threads[i].getState();
        }
    }

    finish = true;
    for (int i = 0; i < 10; i++) {
        finish = finish && (threads[i].getState() == State.TERMINATED);
    }
}
```

Threads - Ejemplo

```
private static void writeThreadInfo(PrintWriter pw, Thread thread, State state) {  
    pw.printf("Main : Id %d - %s\n", thread.getId(), thread.getName());  
    pw.printf("Main : Priority: %d\n", thread.getPriority());  
    pw.printf("Main : Old State: %s\n", state);  
    pw.printf("Main : New State: %s\n", thread.getState());  
    pw.printf("Main : *****\n");  
}
```

Threads - Ejemplo

- Que sucedió con la prioridad?
- Se puede modificar algún Parámetro para verificar su funcionamiento?

```
Problems @ Javadoc Declaration Console Progress
<terminated> Main (49) [Java Application] C:\Program Files\Java\jre-9.0.4\bin\javaw.exe
Minimum Priority: 1
Normal Priority: 5
Maximun Priority: 10
Thread 'My Thread 0': START
Thread 'My Thread 3': START
Thread 'My Thread 5': START
Thread 'My Thread 4': START
Thread 'My Thread 9': START
Thread 'My Thread 2': START
Thread 'My Thread 1': START
Thread 'My Thread 8': START
Thread 'My Thread 7': START
Thread 'My Thread 6': START
Thread 'My Thread 2': END. Number of Primes: 47
Thread 'My Thread 9': END. Number of Primes: 47
Thread 'My Thread 6': END. Number of Primes: 47
Thread 'My Thread 3': END. Number of Primes: 47
Thread 'My Thread 5': END. Number of Primes: 47
Thread 'My Thread 4': END. Number of Primes: 47
Thread 'My Thread 0': END. Number of Primes: 47
Thread 'My Thread 7': END. Number of Primes: 47
Thread 'My Thread 8': END. Number of Primes: 47
Thread 'My Thread 1': END. Number of Primes: 47
```

Threads - Ejemplo

- Analizar el documento LOG, modificar valores de límite de números primos a calcular y obtener conclusiones.

```
Main.java Calculator.java log.txt
3Main : Status of Thread 2 : NEW
4Main : Status of Thread 3 : NEW
5Main : Status of Thread 4 : NEW
6Main : Status of Thread 5 : NEW
7Main : Status of Thread 6 : NEW
8Main : Status of Thread 7 : NEW
9Main : Status of Thread 8 : NEW
10Main : Status of Thread 9 : NEW
11Main : Id 12 - My Thread 0
12Main : Priority: 10
13Main : Old State: NEW
14Main : New State: TERMINATED
15Main : *****
16Main : Id 13 - My Thread 1
17Main : Priority: 1
18Main : Old State: NEW
19Main : New State: RUNNABLE
20Main : *****
21Main : Id 14 - My Thread 2
22Main : Priority: 10
23Main : Old State: NEW
24Main : New State: TERMINATED
25Main : *****
26Main : Id 15 - My Thread 3
27Main : Priority: 1
28Main : Old State: NEW
29Main : New State: RUNNABLE
30Main : *****
```